ENTORNOS DE DESARROLLO

NEREA MONTOYA OLIVARES

ÍNDICE

1. Enunciado.

2. Refactorización

2.1. Las clases deberán formar parte del paquete cuentas.

2.2. Cambiar el nombre de la variable "miCuenta" por "cuenta1".

2.3. Introducir el método operativa_cuenta, que englobe las sentencias de la clase Main que operan con el objeto cuenta1.

2.4. Encapsular los atributos de la clase CCuenta.

2.5. Añadir parámetro al método operativa_cuenta, de nombre cantidad y de tipo float.

3. GIT

3.1. Configurar GIT para el proyecto. Crear un repositorio público en GitHub.

3.2. Realizar, al menos, una operación commit (sube el proyecto al reposirotio remoto), comentando el resultado de la ejecución.

3.3. Mostrar el historial de versiones para el proyecto mediante un comando desde consola.

4. JAVADOC

4.1. Insertar comentarios JavaDoc en la clase CCuenta.

4.2. Generar documentación JavaDoc para todo el proyecto y comprueba que abarca todos los métodos y atributos de la clase CCuenta.

REFERENCIAS:

-Diario de programación:
https://blog.chuidiang.org/2007/02/20/refactoring-con-eclipse/#:~:text=Sobre la clase le damos al botón derecho,y el nombre de la clase donde aparezca.

-Youtube: (Videos en general).

Enunciado

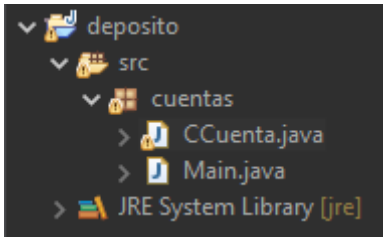
En el proyecto **Java "Deposito"**, hay definida una Clase llamada *CCuenta*, que tiene una serie de atributos y métodos. El proyecto cuenta asimismo con una Clase *Main*, donde se hace uso de la clase descrita.

El código fuente necesario para la actividad se encuentra adjunto ("Deposito.rar"). Basándonos en ese proyecto, vamos a realizar las siguientes actividades

REFACTORIZACIÓN

1. Las clases deberán formar parte del paquete cuentas.

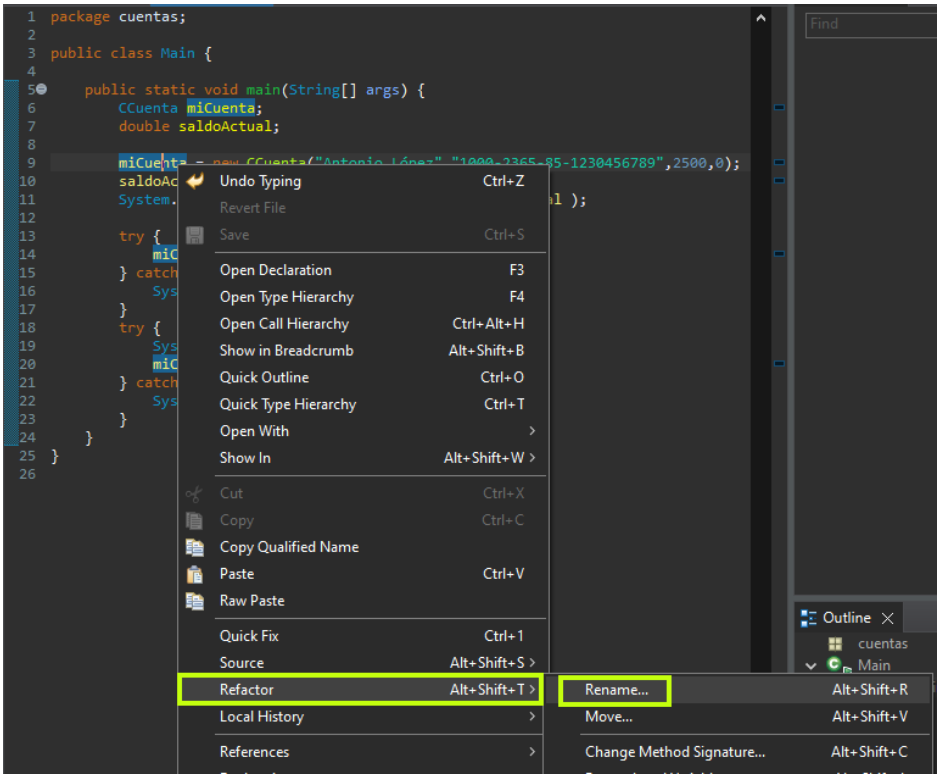
Las clases se ubican dentro del package **cuentas**.



2. Cambiar el nombre de la variable "miCuenta" por "cuenta1".

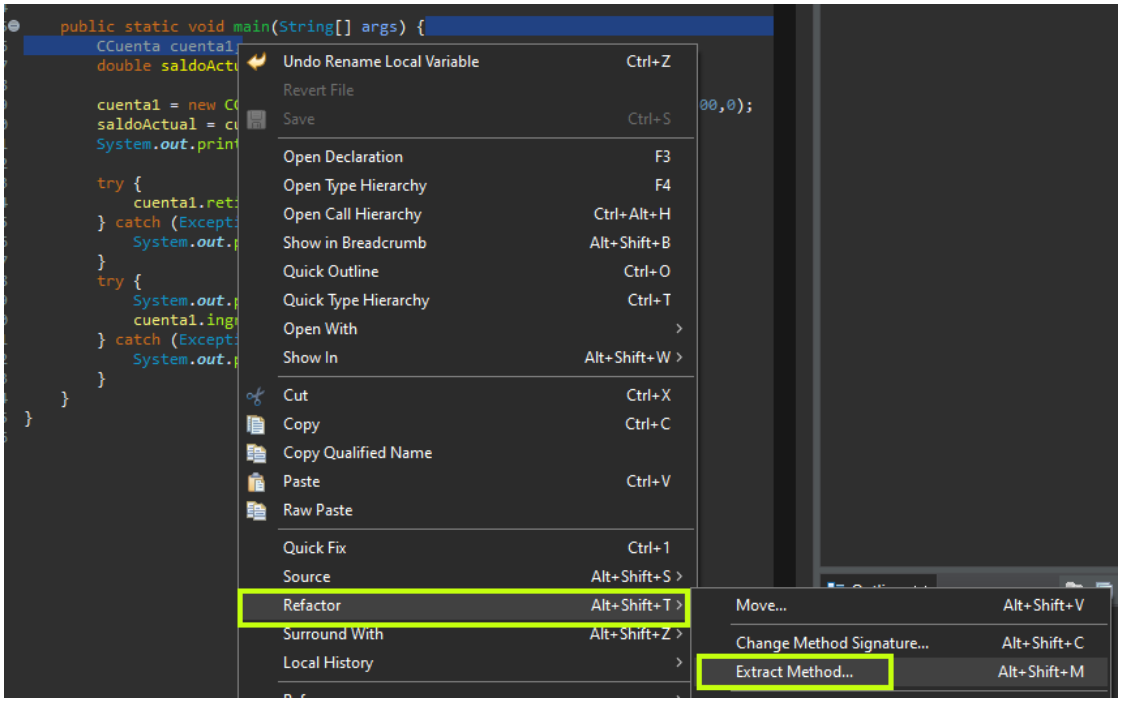
Seleccionamos **“miCuenta”** (click derecho)→ *Refactor*→ *Rename*.

(Cambiamos **miCuenta** por **cuenta1 d**e manera automática en todo el código.)

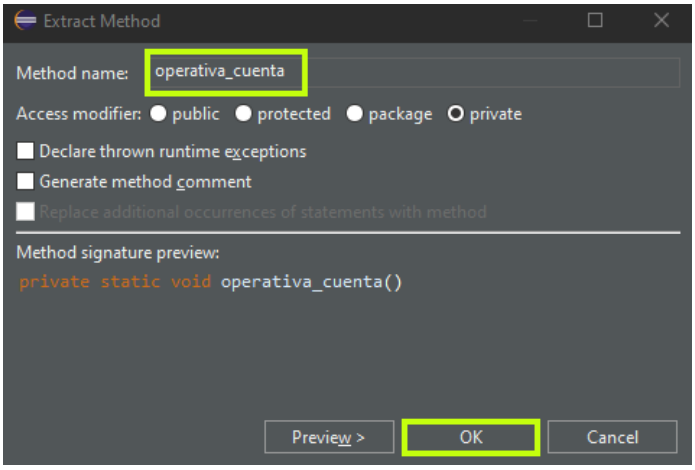


3. Introducir el método operativa_cuenta, que englobe las sentencias de la clase Main que operan con el objeto cuenta1.

Seleccionamos las líneas de código (click derecho)→ *Refactor*→ *Extract Method*...



Añadir **operativa_cuenta** al método → OK.





```
package cuentas;

public class Main {

    public static void main(String[] args) {
        operativa_cuenta();

        cuenta1 = new CCuenta("Antonio López","1000-2365-85-1230456789",2500,0);
        saldoActual = cuenta1.estado();
        System.out.println("El saldo actual es"+ saldoActual );

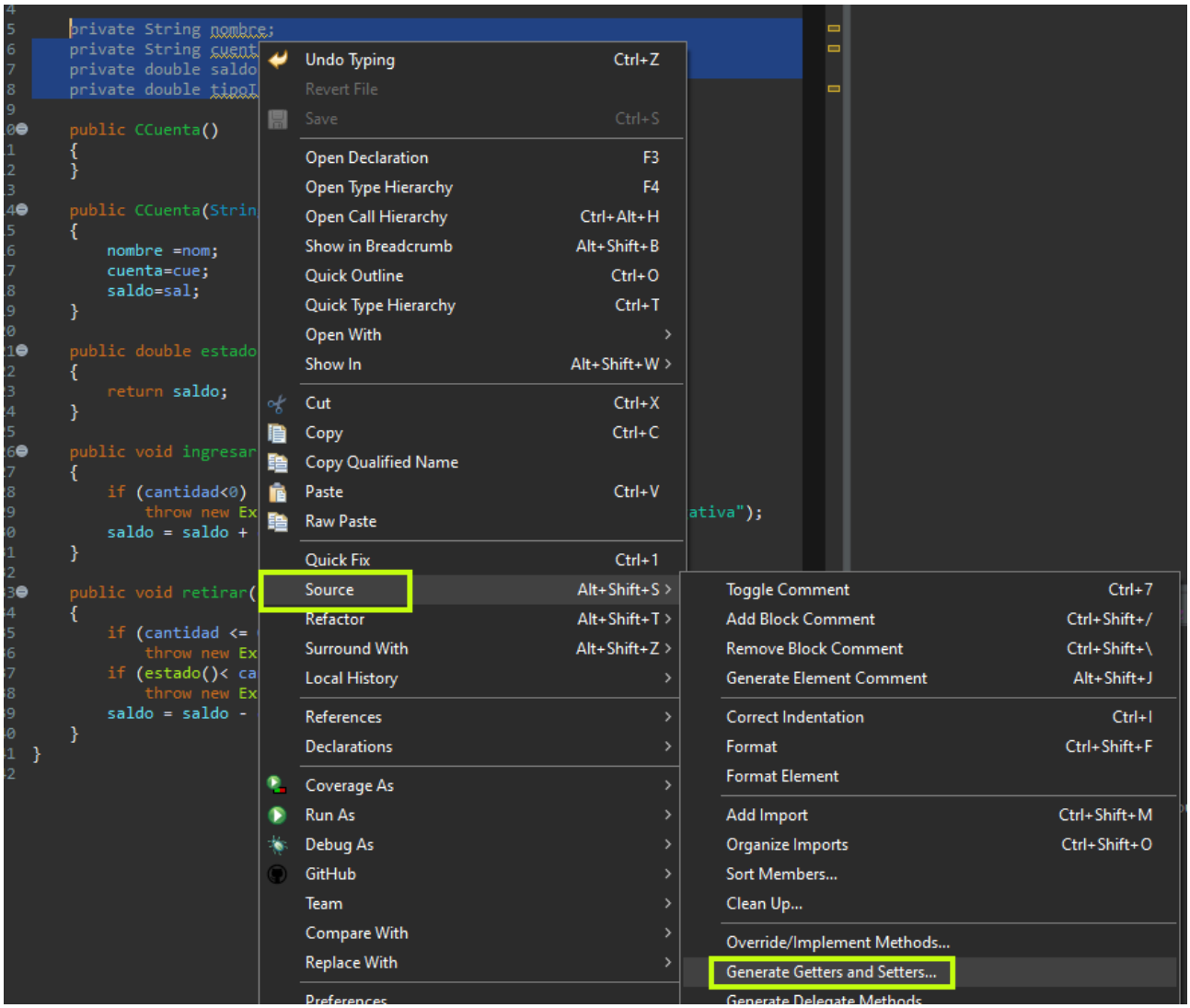
        try {
            cuenta1.retirar(2300);
        } catch (Exception e) {
            System.out.print("Fallo al retirar");
        }
        try {
            System.out.println("Ingreso en cuenta");
            cuenta1.ingresar(695);
        } catch (Exception e) {
            System.out.print("Fallo al ingresar");
        }
    }

    private static void operativa_cuenta() {
        CCuenta cuenta1;
    }
}
```

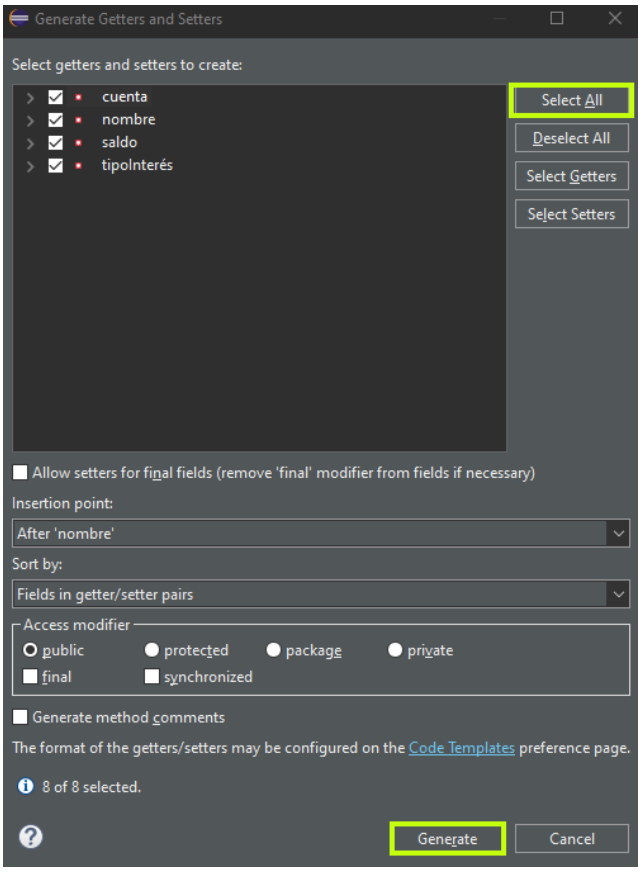
Resultado del método operativa_cuenta

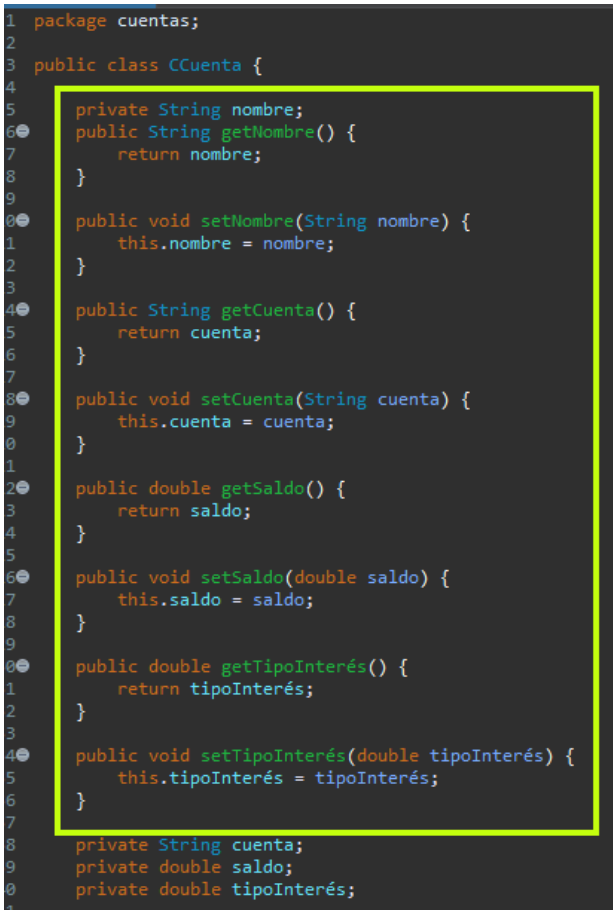
4. Encapsular los atributos de la clase CCuenta.

Seleccionar atributos (click derecho)→ *Source*→ *Generate Getters and Setters*.



Sellect All→ Generate





```java
package cuentas;

public class CCuenta {

    private String nombre;
    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getCuenta() {
        return cuenta;
    }

    public void setCuenta(String cuenta) {
        this.cuenta = cuenta;
    }

    public double getSaldo() {
        return saldo;
    }

    public void setSaldo(double saldo) {
        this.saldo = saldo;
    }

    public double getTipoInterés() {
        return tipoInterés;
    }

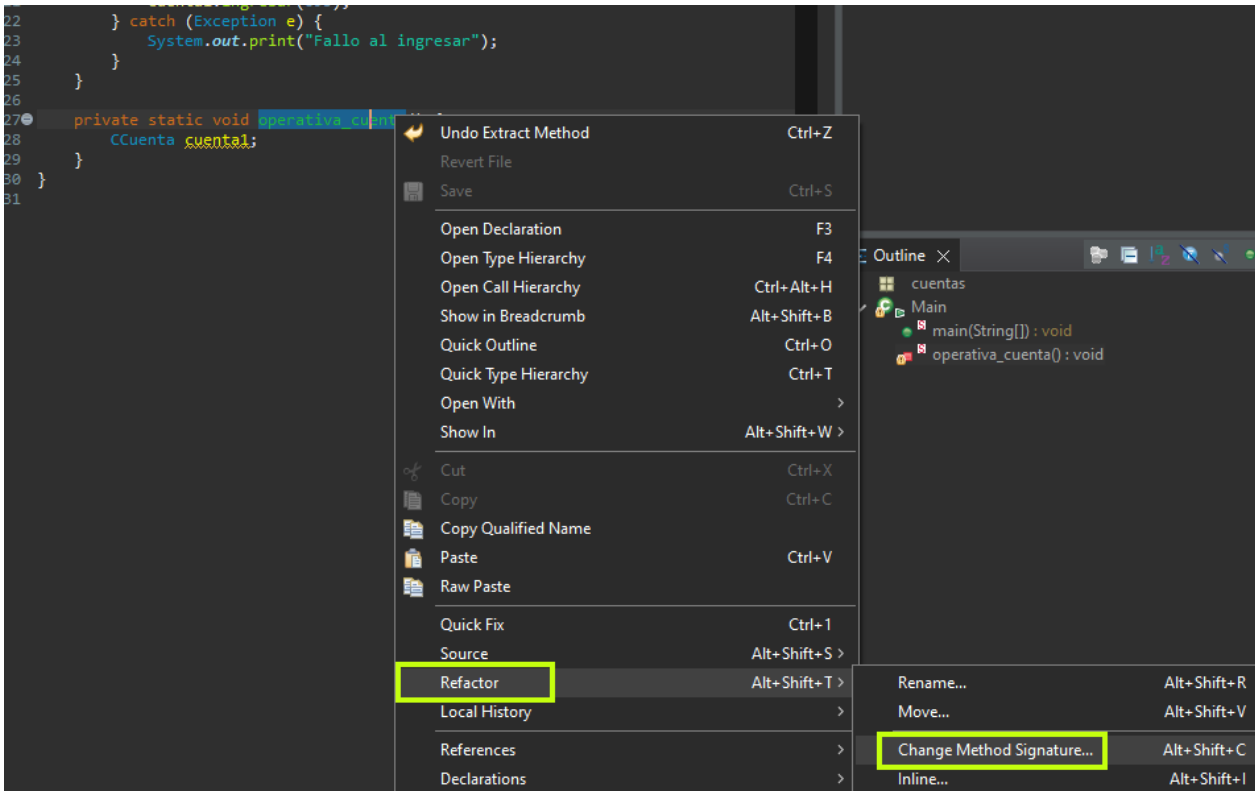
    public void setTipoInterés(double tipoInterés) {
        this.tipoInterés = tipoInterés;
    }

    private String cuenta;
    private double saldo;
    private double tipoInterés;
```

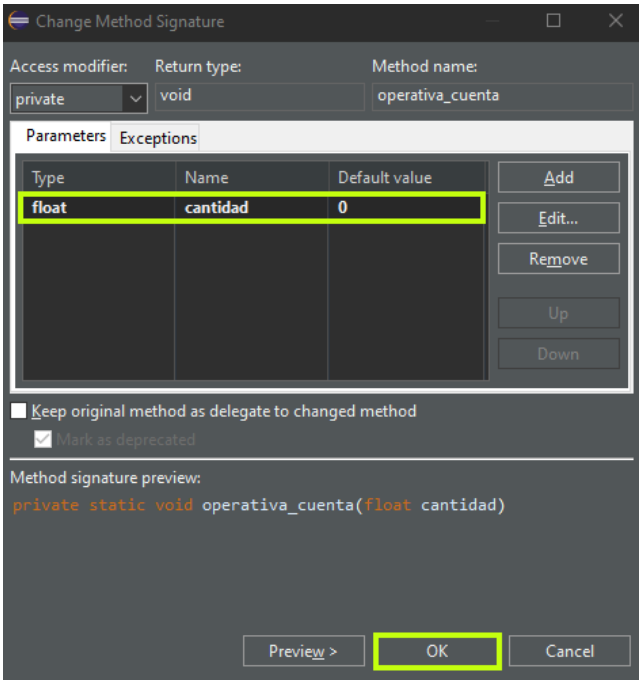
Resultado de encapsulación de atributos

5. Añadir un nuevo parámetro al método operativa_cuenta, de nombre cantidad y de tipo float.

En Main, seleccionamos **operativa_cuenta** → *Refactor* → *Change Method Signature*.



Añadimos el tipo **float** de nombre cantidad y valor por defecto 0.




```java
package cuentas;

public class Main {

    public static void main(String[] args) {
        CCuenta cuenta1;
        operativa_cuenta(0);
        double saldoActual;

        cuenta1 = new CCuenta("Antonio López","1000-2365-85-1230456789",2500,0);
        saldoActual = cuenta1.estado();
        System.out.println("El saldo actual es"+ saldoActual );

        try {
            cuenta1.retirar(2300);
        } catch (Exception e) {
            System.out.print("Fallo al retirar");
        }
        try {
            System.out.println("Ingreso en cuenta");
            cuenta1.ingresar(695);
        } catch (Exception e) {
            System.out.print("Fallo al ingresar");
        }
    }

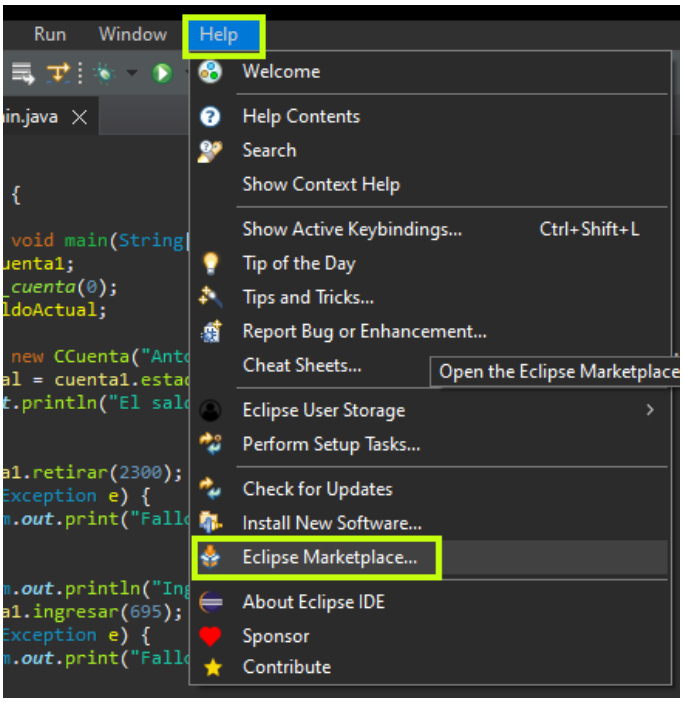
    private static void operativa_cuenta(float cantidad) {
        CCuenta cuenta1;
    }
}
```

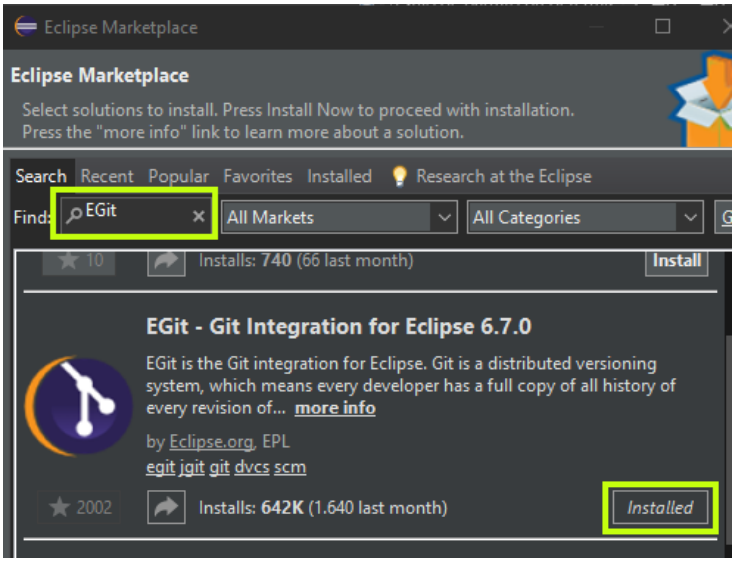
Resultado de añadir parámetro operativa_cuenta

GIT

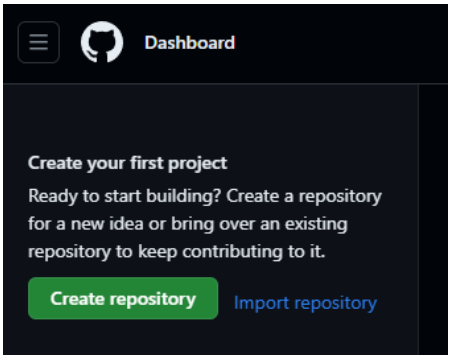
1. Configurar GIT para el proyecto. Crear un repositorio público en GitHub.

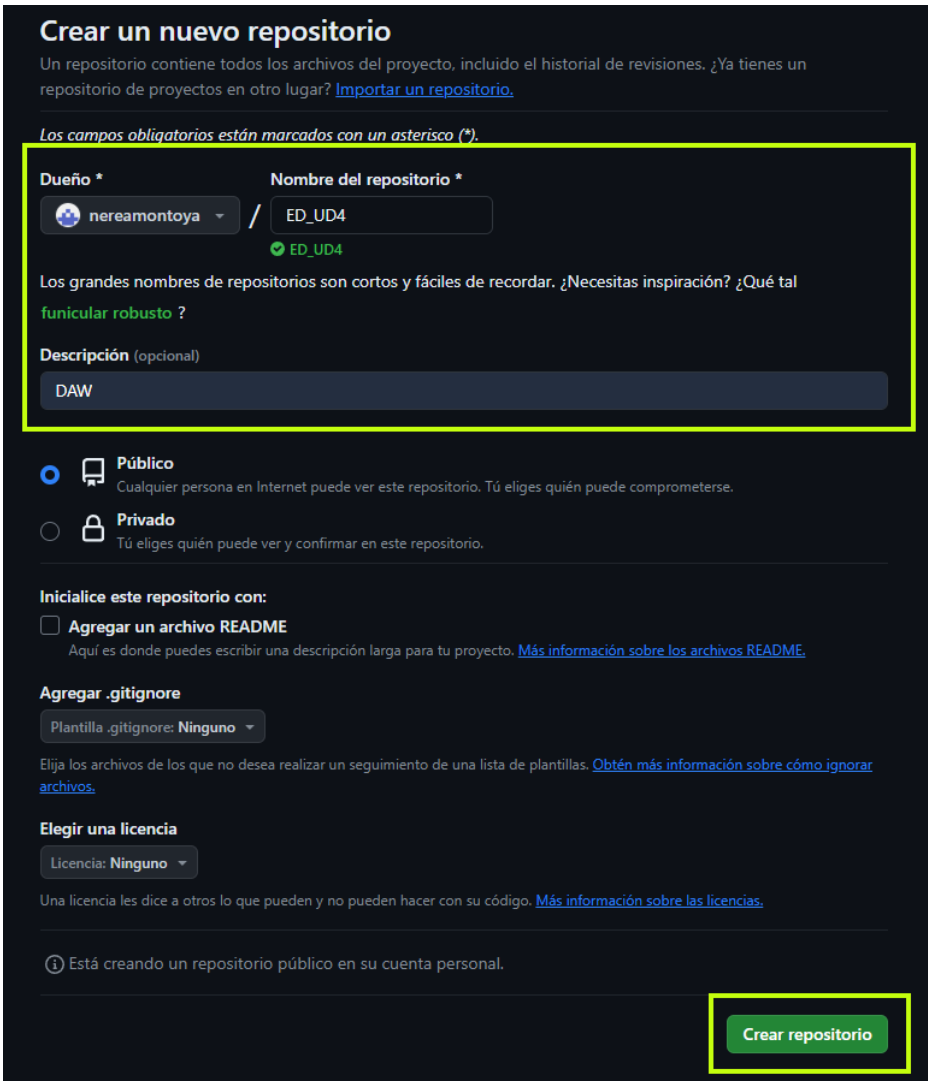
Help → Eclipse Marketplace→ EGit (ya instalado)





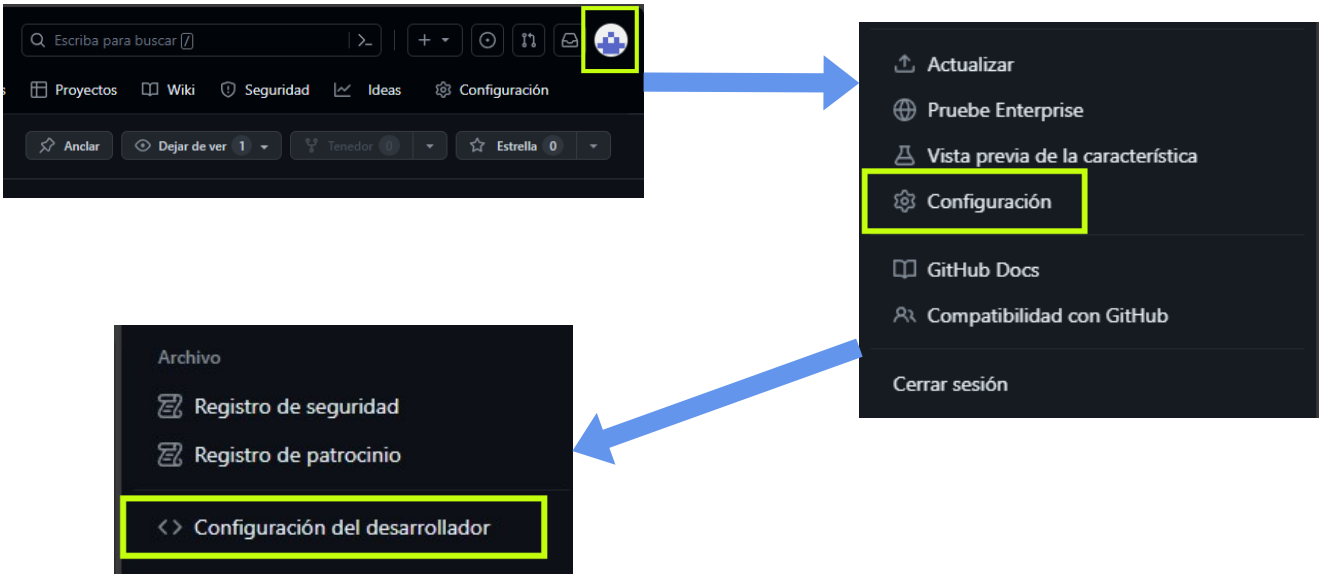
Una vez tengamos cuenta en GitHub crearemos un repositorio.



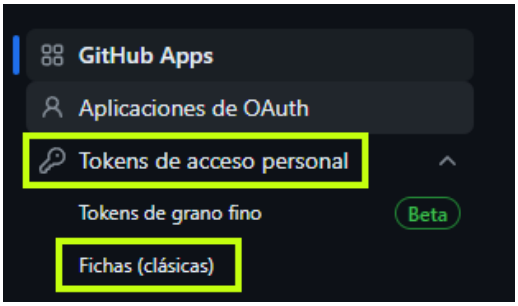


Debemos generar un Token (contraseña temporal para subir el proyecto).

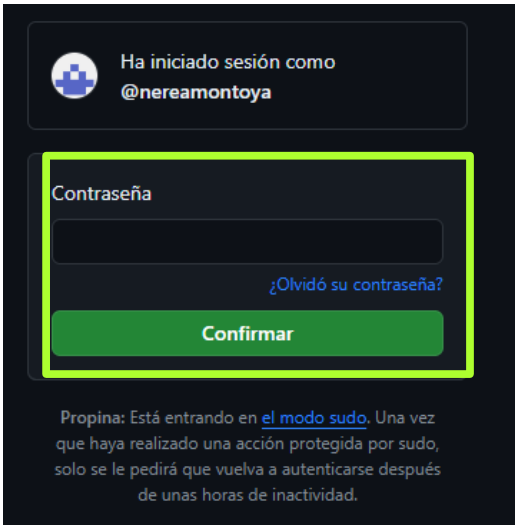
Icono de usuario de GitHub→ *Configuración*→ *Configuración del desarrollador.*



Tokens de acceso personal → Fichas (clásicas) → Generar nuevo token → contraseña.



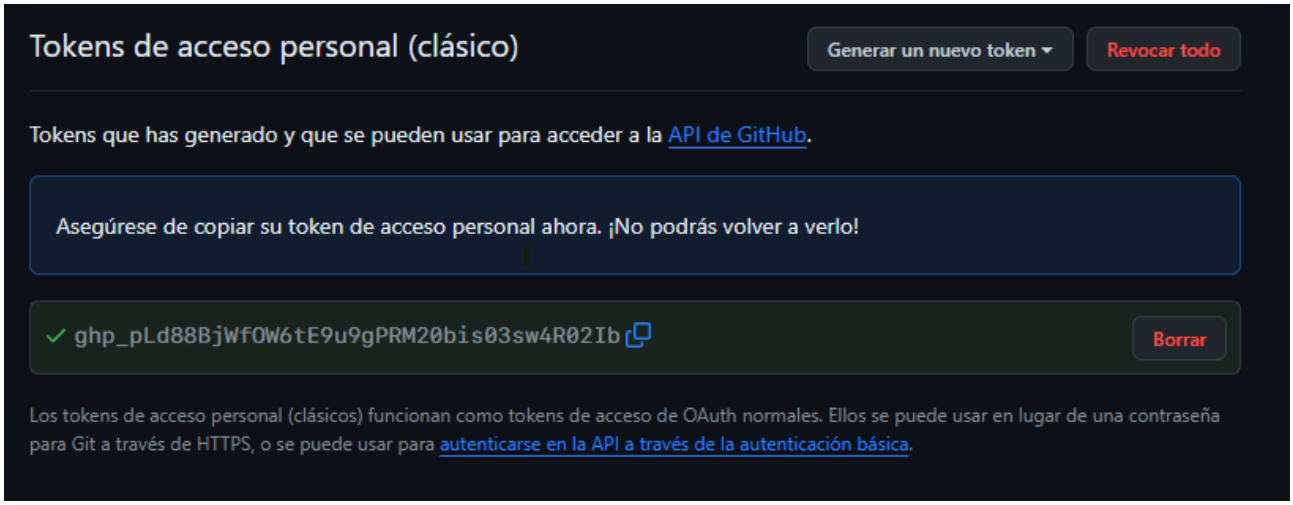




Añadimos nota → tiempo de expiración → Acceso → Generar token.

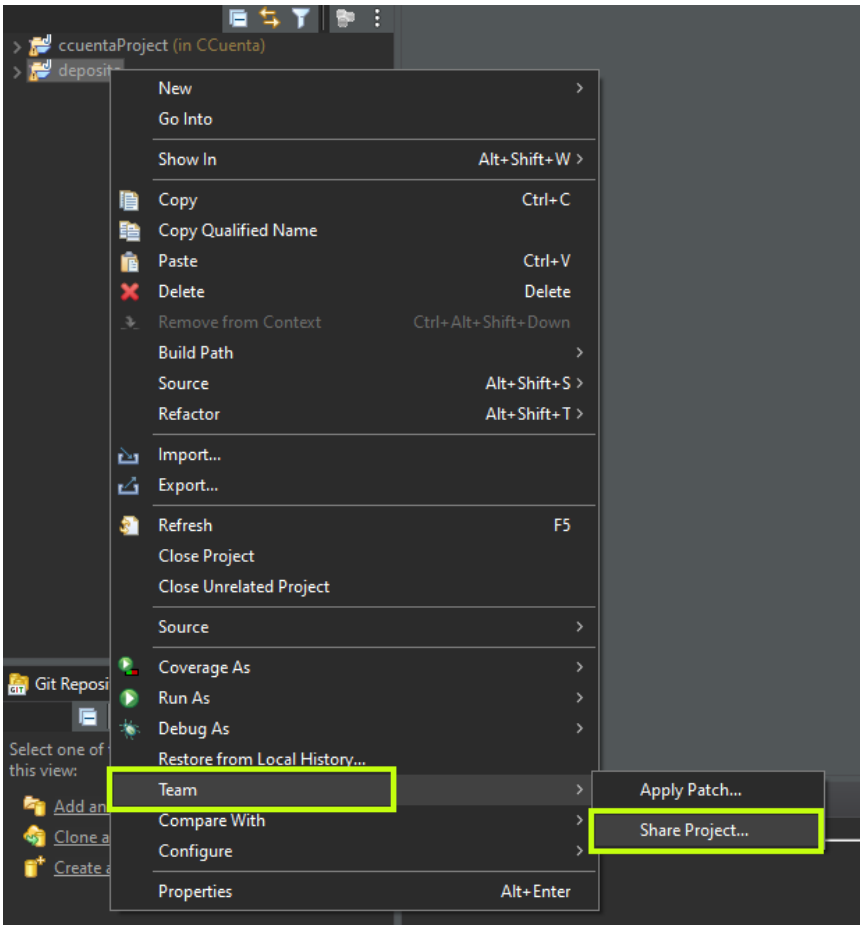


Ya tenemos Token hecho, guardamos la contraseña en algún bloc de notas.

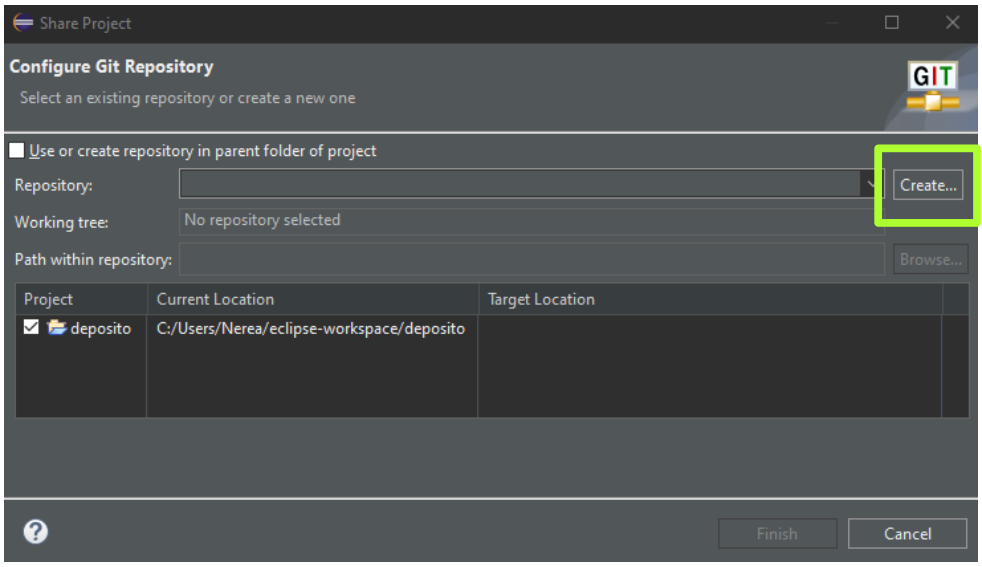


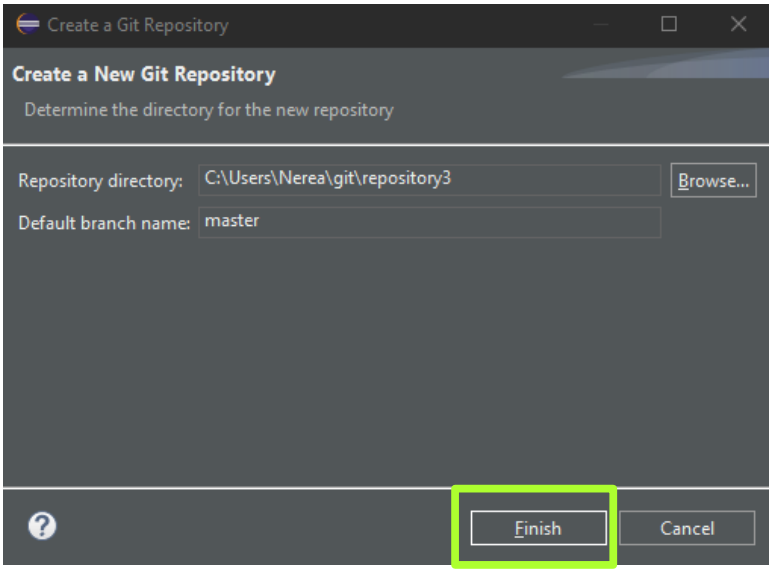
Ahora volvemos a Eclipse.

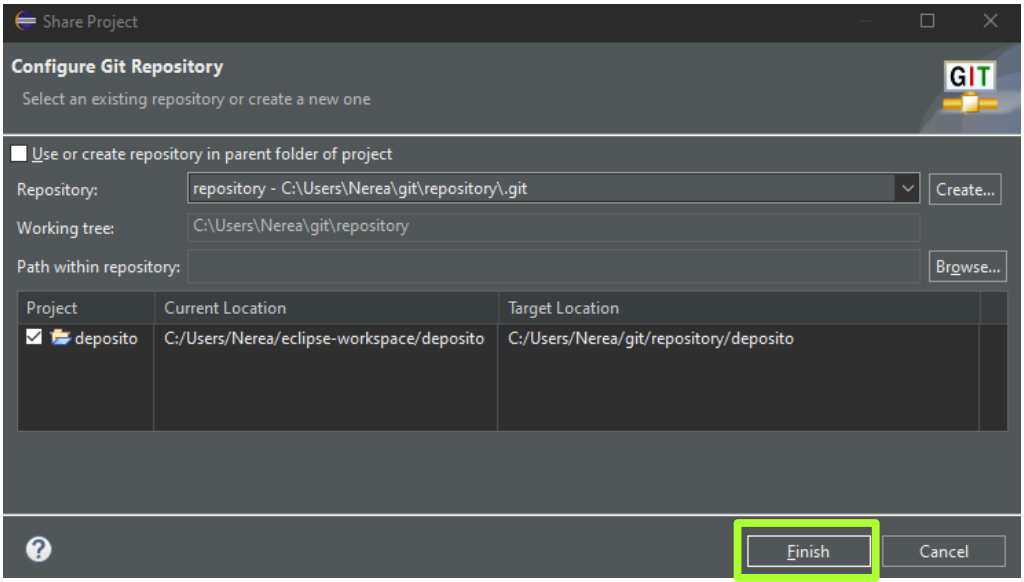
Botón derecho en proyecto **deposito**→ *Team* → *Share project*.



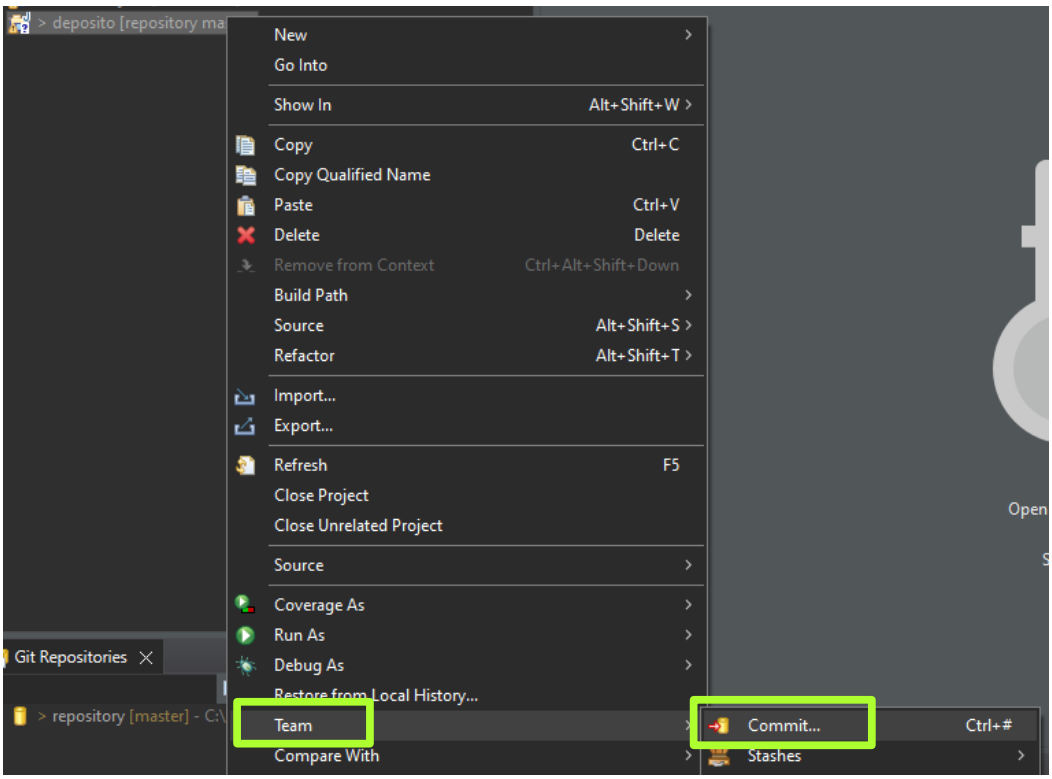
Create→ Elegir repositorio→ Finish



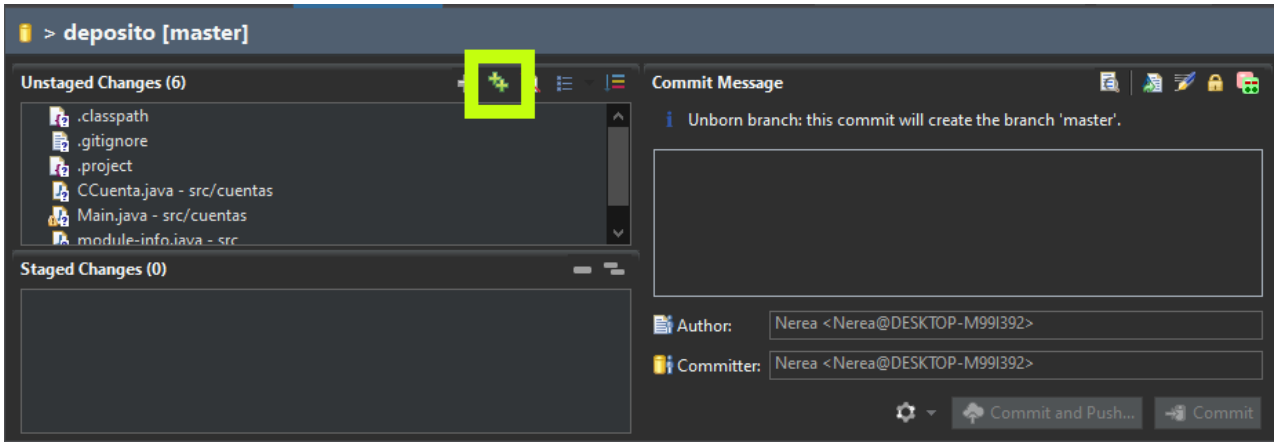




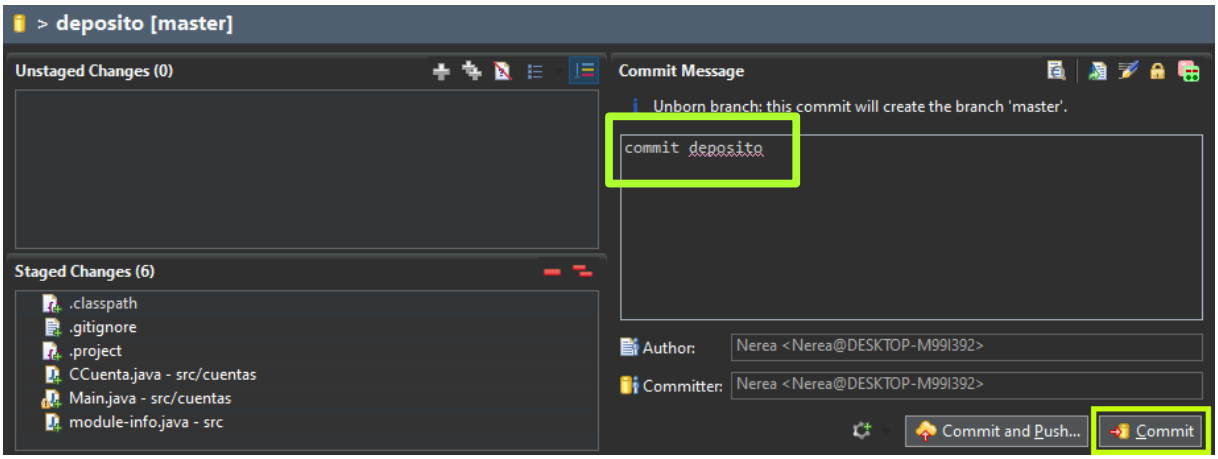
Click derecho en proyecto → Team → Commit.



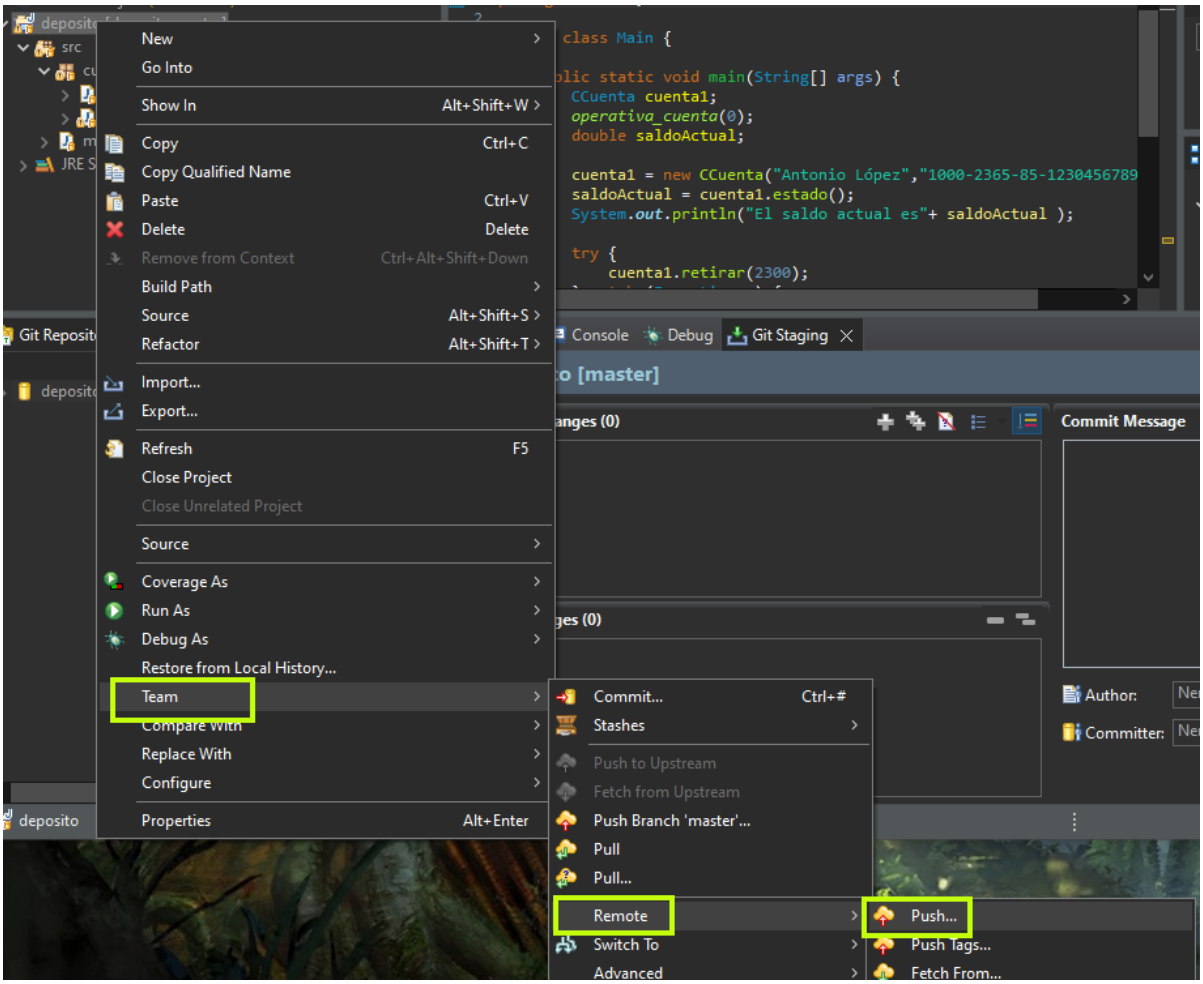
Pulsamos ++ → Se añadirá todo a *Staged Changes.*



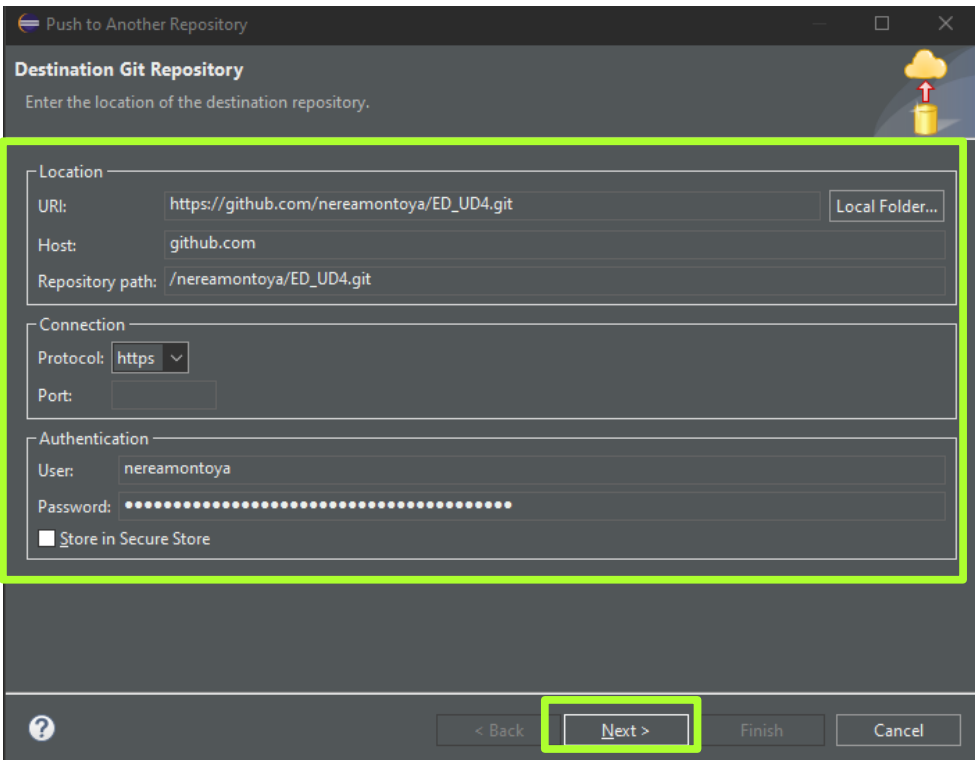
Escribimos nuestro commit y pulsamos commit.



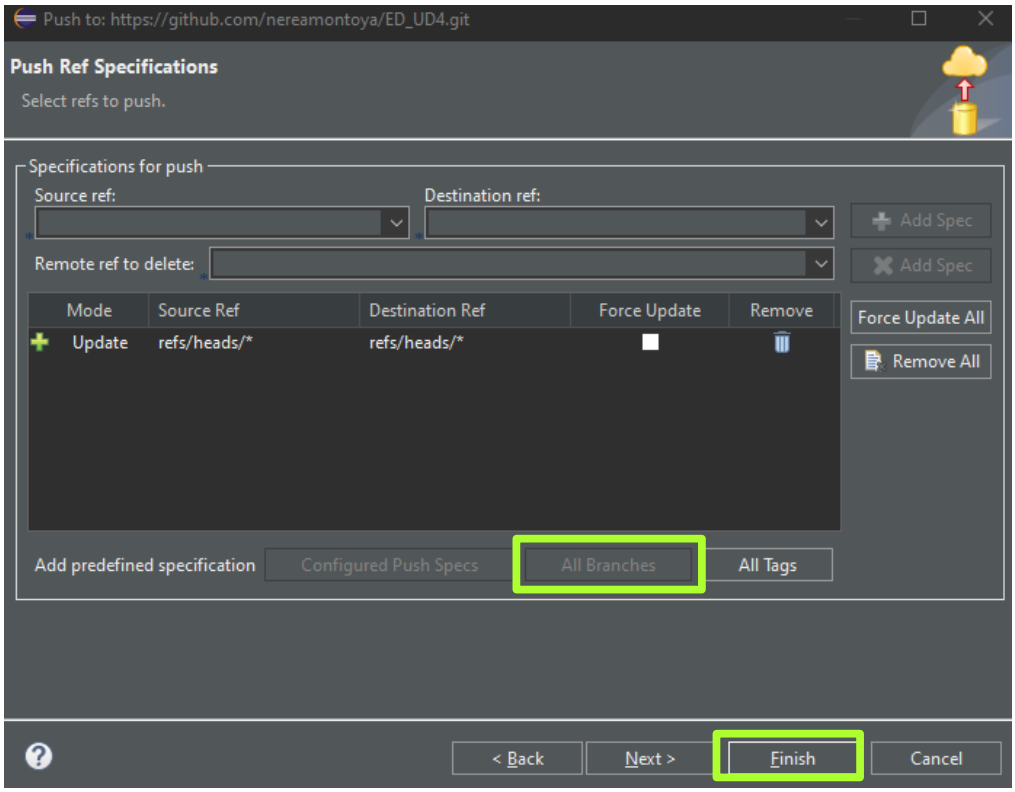
Click dercho **deposito** → *Team* → *Remote* → *Push.*



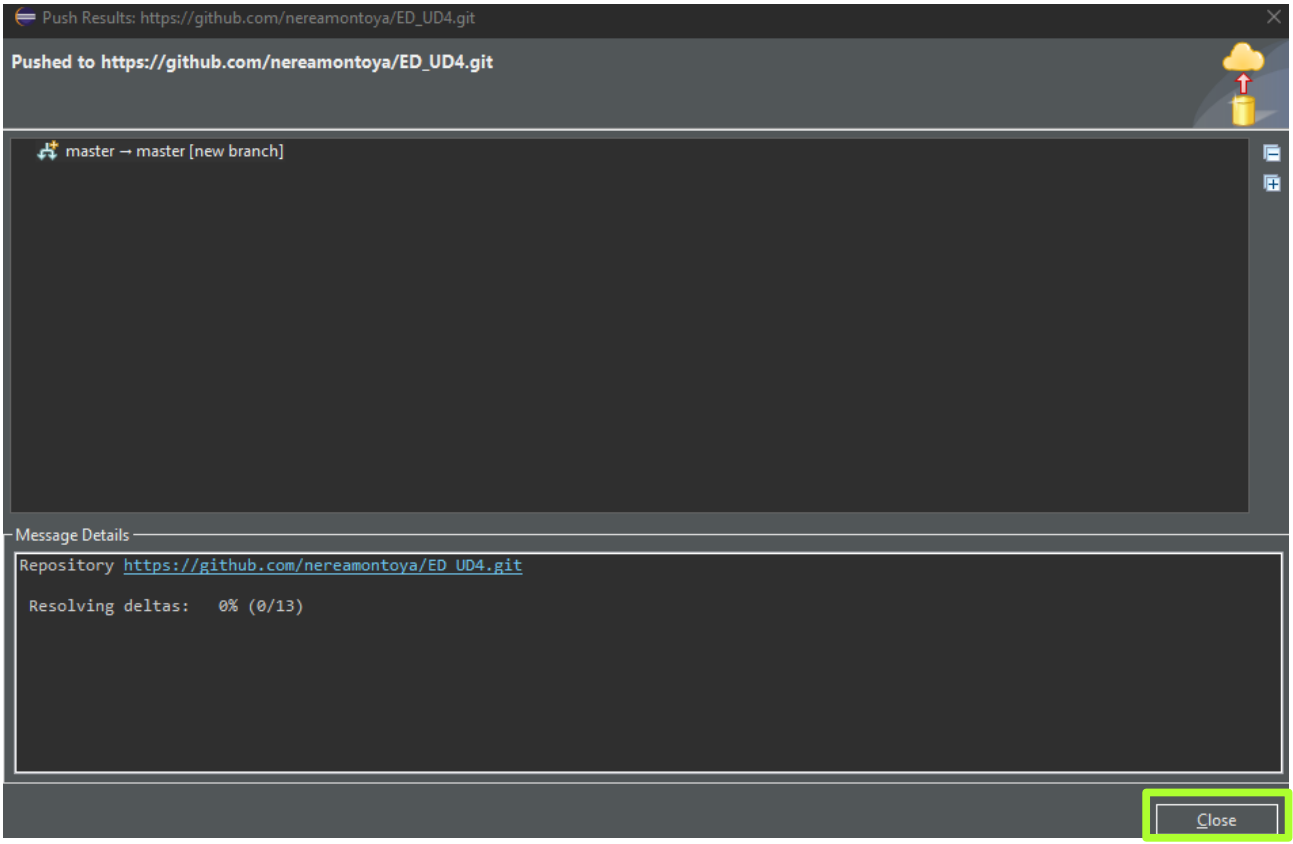
Copiamos URL repositorio GitHub, usuario y token → *Next.*



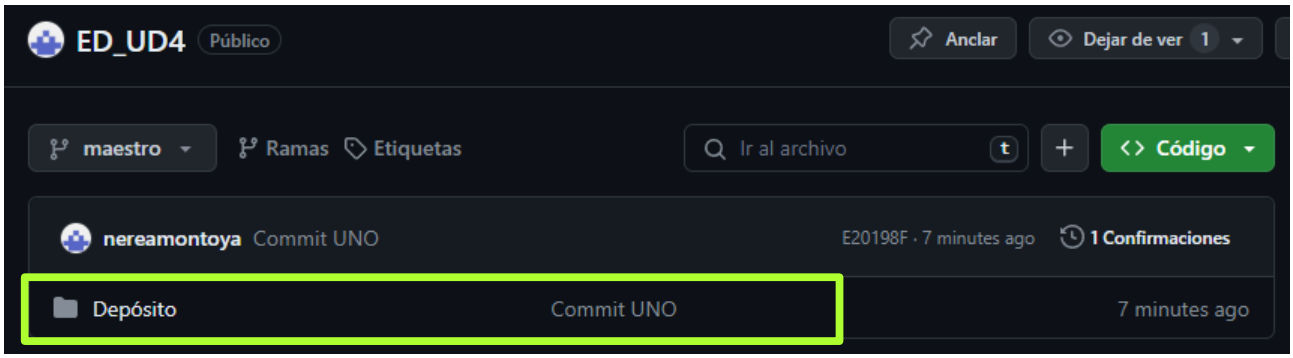
All Branches → Finish

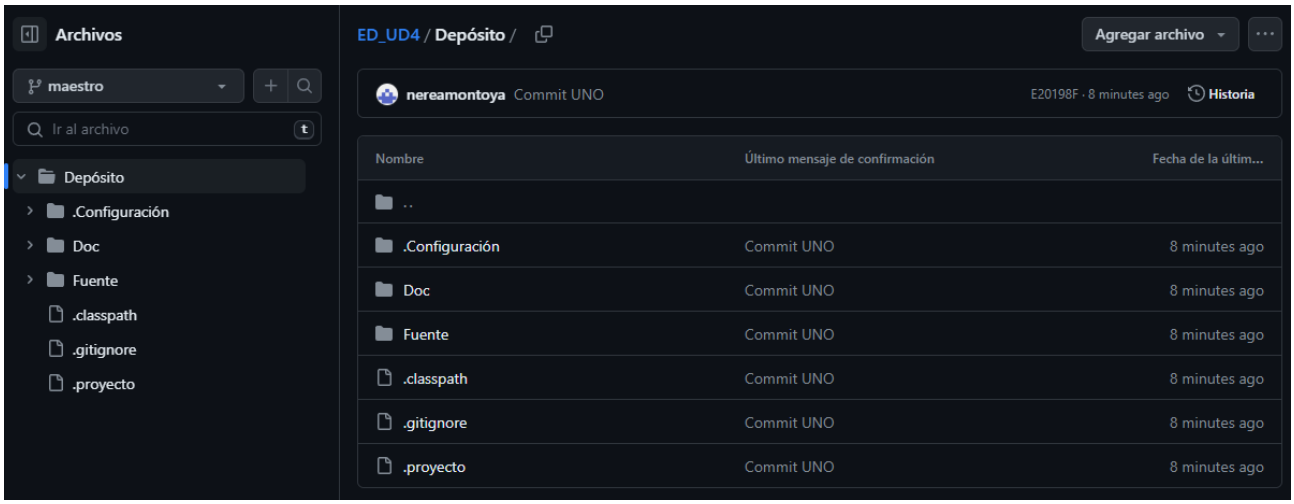


Nos dice que se ha subido el proyecto a GitHub y cerramos.

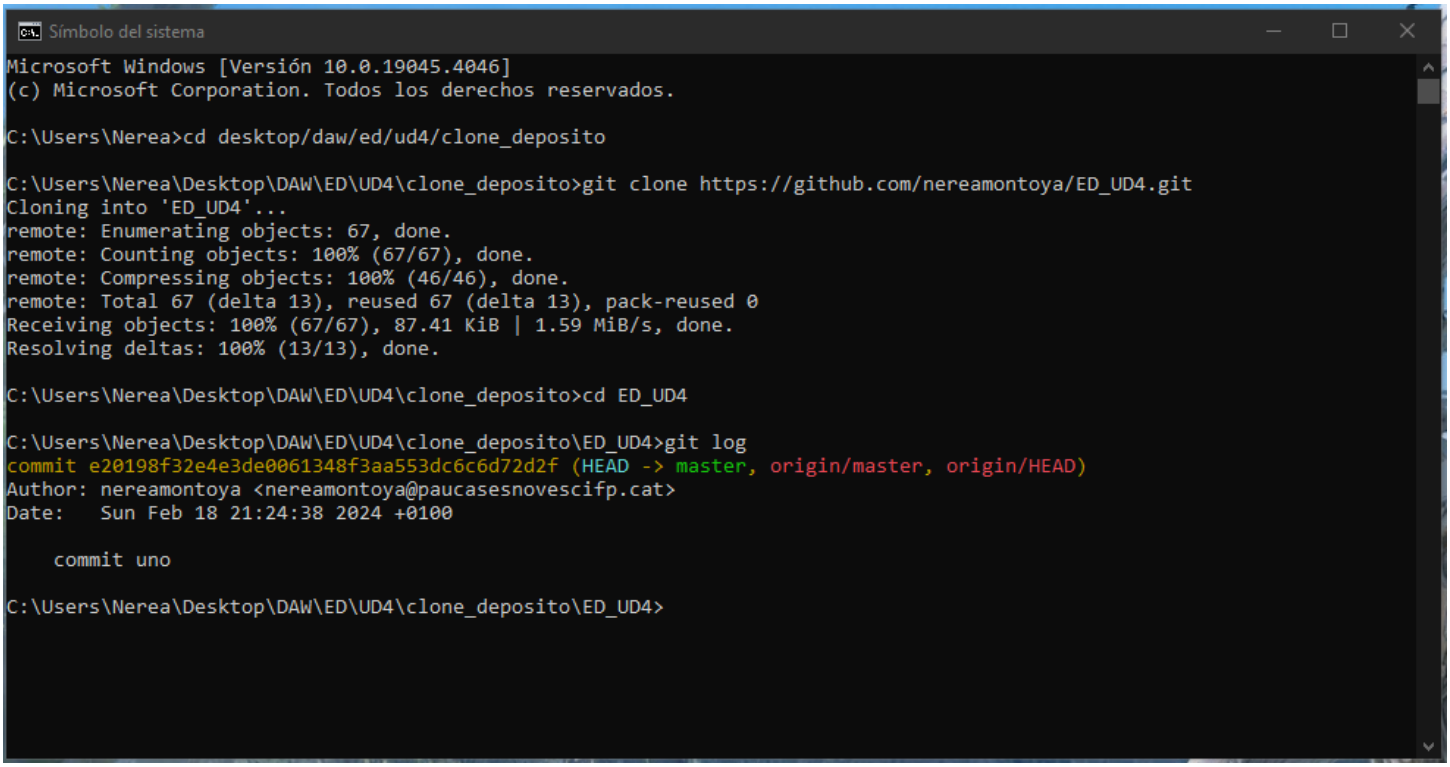


Aquí podemos ver que se ha subido el proyecto **deposito** a GitHub.





3. Mostrar el historial de versiones para el proyecto mediante un comando desde consola.



JAVADOC

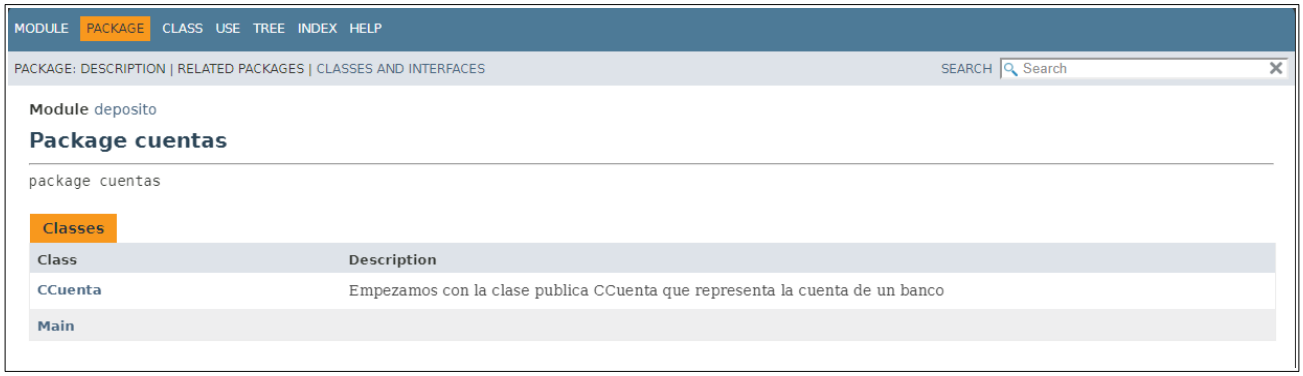
1. Insertar comentarios JavaDoc en la clase CCuenta.

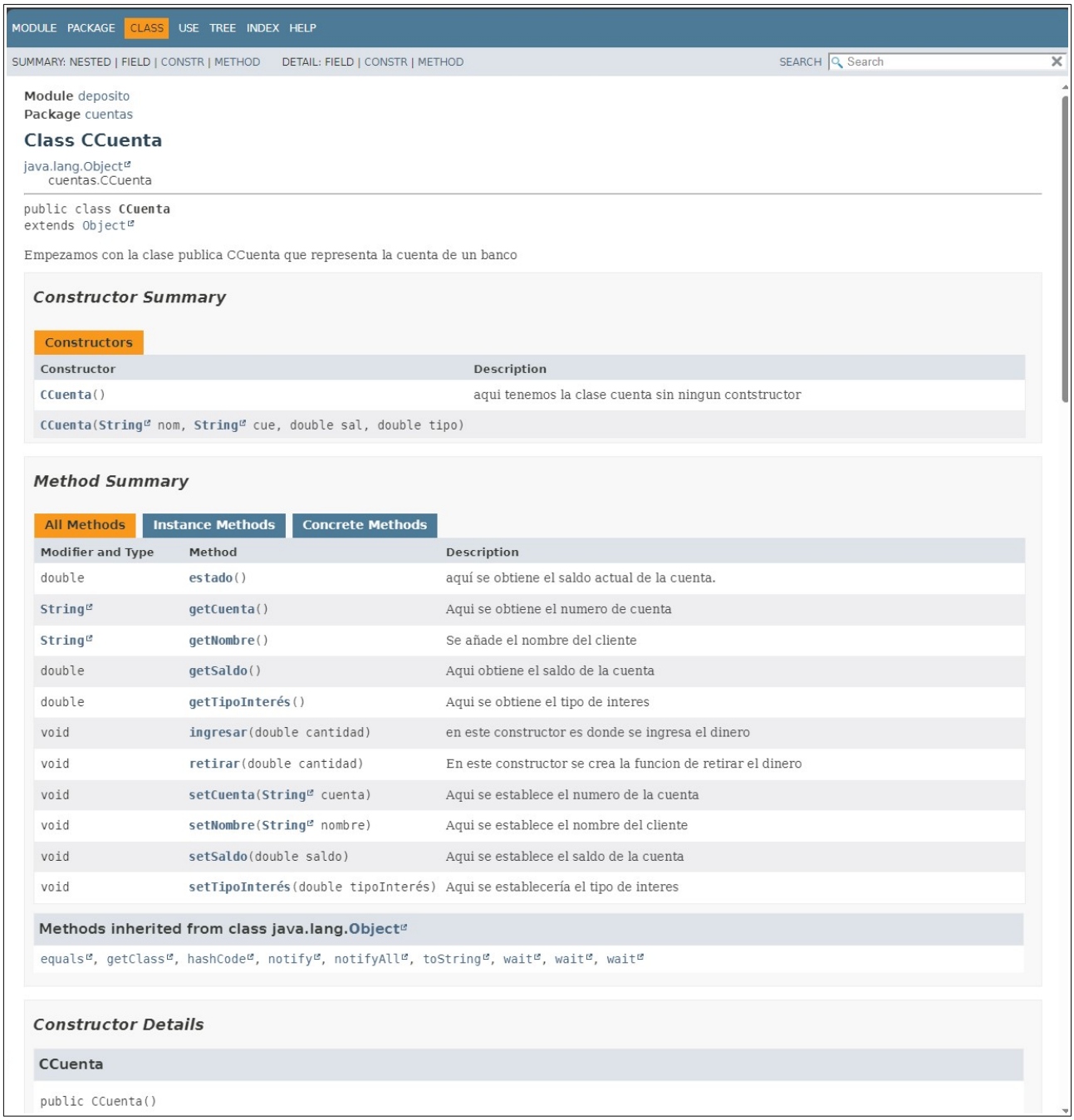
*CCuenta.java Main.java

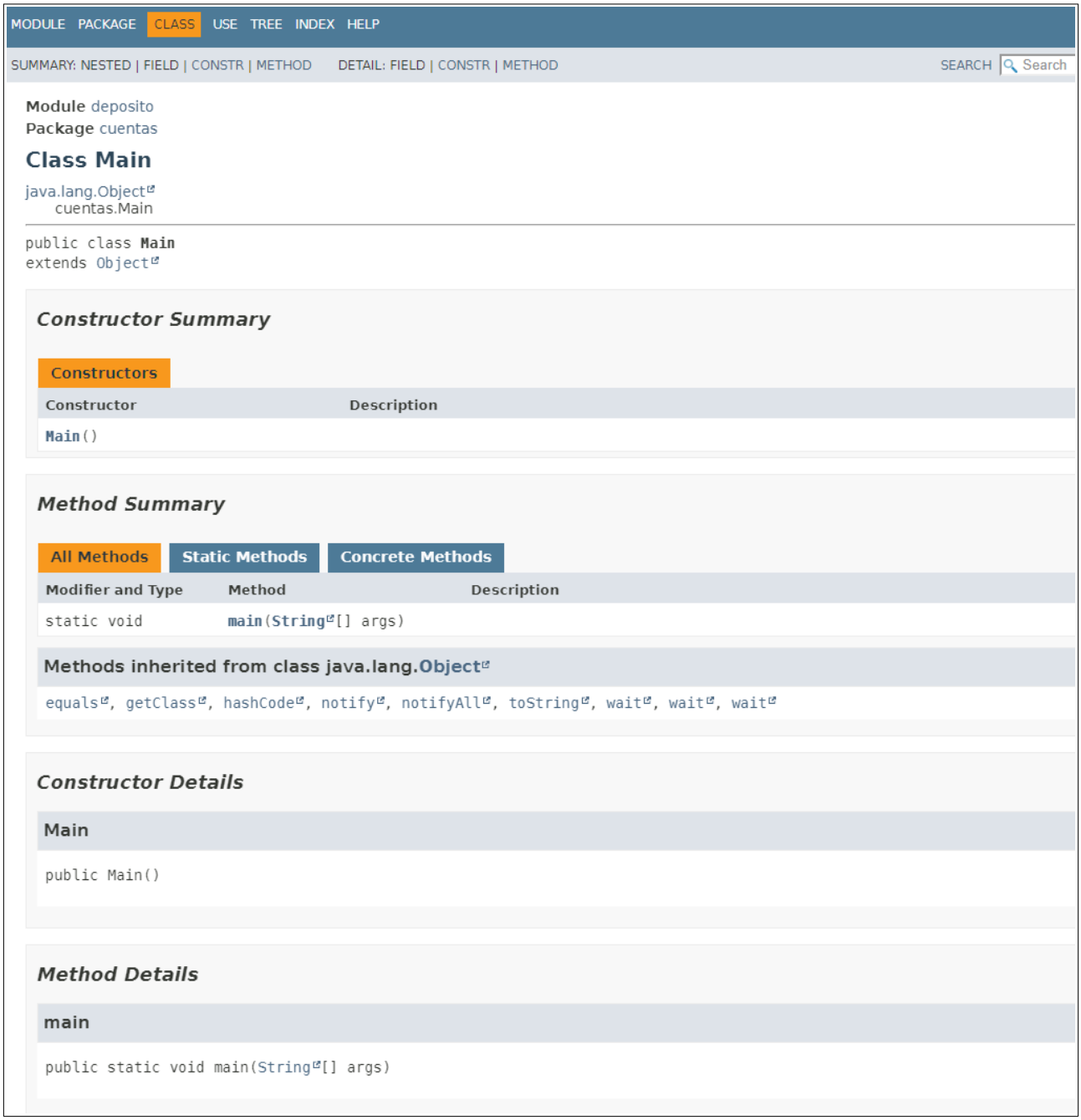
```java
package cuentas;
/** Empezamos con la clase publica CCuenta que representa la cuenta de un banco */
public class CCuenta {
    /**
     * Se añade el nombre del cliente
     * @return : devuelve el nombe de la cuenta
     */
    public String getNombre() {
        return nombre;
    }
    /**
         * Aqui se establece el nombre del cliente
         * @param nombre : nombre del cliente
         */
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    /**
         * Aqui se obtiene el numero de cuenta
         * @return : numero de cuenta
         */
    public String getCuenta() {
        return cuenta;
    }
    /**
         * Aqui se establece el numero de la cuenta
         * @param cuenta : esto es el numero de la cuenta.
         */
    public void setCuenta(String cuenta) {
        this.cuenta = cuenta;
    }
    /**
     * Aqui obtiene el saldo de la cuenta
     * @return : devuelve el saldo de la cuenta
     */
    public double getSaldo() {
        return saldo;
    }
     /**
     * Aqui se establece el saldo de la cuenta
     * @param saldo: esto dice el saldo de la cuenta
     */
    public void setSaldo(double saldo) {
        this.saldo = saldo;
    }
    /**
         * Aqui se obtiene el tipo de interes
         * @return : devuelve el tipo de interes.
         */
    public double getTipoInterés() {
        return tipoInterés;
    }
    /**
     * Aqui se estableceria el tipo de interes
     * @param tipoInterés : esto dice el tipo de interes.
     */
    public void setTipoInterés(double tipoInterés) {
        this.tipoInterés = tipoInterés;
    }
```

```java
    /** Aqui creamos el constructor de la clase CCuenta */
    private String nombre;
    private String cuenta;
    private double saldo;
    private double tipoInterés;
    /** aqui tenemos la clase cuenta sin ningun contstructor */
    public CCuenta(){
    }
    /** @param nom : para el nombre del titular de la cuenta
     * @param cue: para el numero de cuenta del titular
     * @param sal: el Saldo incial que tiene el titular
     * @param tipo: el interés que genera la cuenta
     */
    public CCuenta(String nom, String cue, double sal, double tipo)
    {
        nombre =nom;
        cuenta=cue;
        saldo=sal;
    }
    /** aquí se obtiene el saldo actual de la cuenta.
    *
    * @return : devuelve el saldo actual de la cuenta
    */
    public double estado()
    {
        return saldo;
    }
    /**
     * en este constructor es donde se ingresa el dinero
     * @param cantidad : el dinero que se ingresa
     * @throws Exception : Si se deposita una cantidad negativa
     */
    public void ingresar(double cantidad) throws Exception
    {
        if (cantidad<0)
            throw new Exception("No se puede ingresar una cantidad negativa");
        saldo = saldo + cantidad;
    }
    /**
     *En este constructor se crea la funcion de retirar el dinero
     * @param cantidad: el dinero que se retira
     * @throws Exception : si se retira una cantidad negativa o no hay suficiente saldo
     */
    public void retirar(double cantidad) throws Exception
    {
        if (cantidad <= 0)
            throw new Exception ("No se puede retirar una cantidad negativa");
        if (estado()< cantidad)
            throw new Exception ("No se hay suficiente saldo");
        saldo = saldo - cantidad;
    }
}
```

2. Generar documentación JavaDoc para todo el proyecto y comprueba que abarca todos los métodos y atributos de la clase CCuenta.







MODULE PACKAGE CLASS USE TREE INDEX HELP

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD SEARCH Search

Module deposito
Package cuentas

Class Main

java.lang.Object
cuentas.Main

```
public class Main
extends Object
```

Constructor Summary

Constructors

Constructor	Description
Main()	

Method Summary

All Methods | Static Methods | Concrete Methods

Modifier and Type	Method	Description
static void	main(String[] args)	

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

Main

```
public Main()
```

Method Details

main

```
public static void main(String[] args)
```

Link al repositorio de GitHub:

https://github.com/nereamontoya/ED_UD4.git